

Reviewer Pack — Minimal Order of Geometric Group Actions and SAT Clause Subs...

Entry 8427638050a8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 21:35:18 UTC

Minimal Order of Geometric Group Actions and SAT Clause Subset Entropy Correlation

Entry ID: 8427638050a8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 21:35:18 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory **Field B** (complexity object): Boolean Satisfiability (SAT) Clause Subsets

Statement:

For every d -regular graph G representing a Boolean satisfiability problem, the minimal order of a geometric group action on G is linearly correlated with its SAT clause subset entropy, such that $\log_2(\min_order(G)) = \Theta(\log(\sqrt{d} * \log(SAT_clause_entropy(G))))$.

Rationale (proposer's reasoning):

Geometric group actions provide a natural way to capture symmetries in combinatorial objects. The conjecture leverages this structure to establish a connection with the entropy of clause subsets, which is a measure of information content in Boolean satisfiability problems. This correlation could expose new insights into the complexity of SAT and its connections with group theory.

Taxonomy category: GeometricGroupTheory (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ab268df457c08a3a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman's rank correlation coefficient (ρ) between the minimal order of geometric group actions and SAT clause subset entropy exceeds 0.7 for at least 75% of the seeds, with a mean $\rho \geq 0.6$ across all seeds. The conjecture is falsified if any seed has a $\rho \leq 0.4$ or a mean $\rho < 0.5$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal order geometric group actions AND Boolean satisfiability AND clause subsets - Geometric group theory application in SAT problem with minimal action order - Correlation between d-regular graph entropy and geometric group action order

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if n % d != 0:
            raise ValueError("Graph size must be a multiple of the degree")
        graph = {i: [] for i in range(n)}
        edges_added = set()
        for _ in range(d * n // 2):
            while True:
                u, v = random.sample(range(n), 2)
                if (u, v) not in edges_added and (v, u) not in edges_added:
                    graph[u].append(v)
                    graph[v].append(u)
                    edges_added.add((u, v))
                    break
            return graph

    def geometric_group_action(graph):
        n = len(graph)
        min_order = float('inf')
        for i in range(n):
            action = [graph[(i + j) % n] for j in range(n)]
            order = 1
            visited = set()
            while tuple(action[i]) not in visited:
                visited.add(tuple(action[i]))
                action = [graph[(action[i][j] + j) % n] for j in range(n)]
                order += 1
```

```

        min_order = min(min_order, order)
    return min_order

def sat_clause_subset_entropy(clauses):
    total_clauses = len(clauses)
    entropy = 0
    for clause in clauses:
        if len(clause) > 0:
            p = Fraction(1, 2 ** len(clause))
            entropy += -p * math.log(p, 2)
    return entropy

def generate_sat_instance(n):
    clauses = []
    for i in range(n):
        variables = random.sample(range(n), n // 2)
        clause = [random.choice([-1, 1]) * var for var in variables]
        clauses.append(clause)
    return clauses

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    graph = generate_d_regular_graph(n, 3)
    clauses = generate_sat_instance(n)
    min_order = geometric_group_action(graph)
    entropy = sat_clause_subset_entropy(clauses)
    results.append({
        "n": n,
        "min_order": min_order,
        "entropy": entropy
    })

if len(results) < 30:
    return {
        "metric_name": "Spearman's rank correlation coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

def spearman_rank_correlation(x, y):
    n = len(x)
    rank_x = {x[i]: i + 1 for i in range(n)}
    rank_y = {y[i]: i + 1 for i in range(n)}
    sum_differences_squared = sum((rank_x[x[i]] - rank_y[y[i]]) ** 2 for i in range(n))
    return 1 - (6 * sum_differences_squared) / (n * (n**2 - 1))

x = [result["min_order"] for result in results]
y = [result["entropy"] for result in results]
rho = spearman_rank_correlation(x, y)

return {

```

```

        "metric_name": "Spearman's rank correlation coefficient",
        "metric_value": rho,
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": rho > 0.7,
        "counterexample": "" if rho > 0.4 else "rho <= 0.4"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    rho_values = [result["metric_value"] for result in results if result["metric_value"] is not None]
    support_fraction = sum(1 for rho in rho_values if rho > 0.7) / len(rho_values)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={sum(rho_values)/len(rho_values)} std={math.sqrt(sum((rho - support_fraction)**2 for rho in rho_values)/len(rho_values))}")
    elif any(result["conjecture_holds"] is False and result["counterexample"] == "rho <= 0.4" for result in results):
        print(f"RESULT: FALSIFIED counterexample=\"rho <= 0.4\" first_failing_seed={seeds[rho_values.index(min(rho_values))]}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data_or_unsupported_conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0204f52e.py", line 116, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0204f52e.py", line 71, in run_trial
    graph = generate_d_regular_graph(n, 3)
            ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0204f52e.py", line 23, in generate_d_regular_graph
    raise ValueError("Graph size must be a multiple of the degree")
ValueError: Graph size must be a multiple of the degree

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that it was unable to complete its execution and therefore could not provide the necessary correlation coefficients to support or falsify the conjecture. | next: Re-run the test ensuring that the graph size is a multiple of the degree to avoid errors. If the test passes, re-evaluate the correlation coefficients against the pre-registered criteria.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11960
2	preregistration	ollama_remote	glm4:latest	0	0	10180
3	novelty	ollama_remote	glm4:latest	0	0	18072
4	novelty	ollama_remote	glm4:latest	0	0	8870
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17068
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15300
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14440
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14456
9	judge	ollama_remote	glm4:latest	0	0	12414

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122759 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/8427638050a8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8427638050a8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8427638050a8.tar.gz (if generated)